校园时间管家 CampusTime

技术文档（AI 创新应用挑战赛 · 个人赛提交版）

判定层 / 执行层分离 · Propose/Execute 确定性架构 · AI 协同细粒度开发

项目	CampusTime / 校园时间管家 Agent
定位	校园 AI 服务助手（学生活端 / 驿站商户侧 / 管理控制台）
技术栈	Python + FastAPI + 原生 HTML/JS + 自写轻量 ReAct Agent + DeepSeek
仓库	github.com/zhouyuliang0606/campus-time-agent（Public）
规模	7 个页面 / 39 个 API / 253 个细粒度 commit（截至 2026-09-30） / 30 份 Prompt 快照
自检	tests/run_all.py 全绿：通过 1398 项，失败 0 项（共 17 批）
文档版本	2026-09-30

摘要

校园时间管家是一个把分散的校园信息（课表、待办、快递外卖、知识问答）收口到同一个 AI 助手的项目。它有两条互锁的地基：

- Propose / Execute 分离：Agent 只产出「提案」（read-only 工具），学生没在确认卡上点确认之前，系统一个字节都不写库；确认卡的显示由系统盖章的 kind 标记驱动，而非模型自觉。
- 判定层 / 执行层分离：把「听懂人话」和「动手执行」彻底切开。判定层（样本库 → 语义层 → 关键词兜底）负责把一句话听懂并归类，可以换、可以兜底、可以交给大模型；执行层（查空档、排期、出卡、写库）由系统独占，模型永远碰不到写入接口。

本文档说明系统架构、这两条核心设计的成因与落地、2026-09-26 / 09-27 / 09-30 三次架构级迭代，以及 AI 协同开发与交付流程。全部代码以细粒度 commit 提交，开发对话以 Prompt 快照形式随代码入库，可逐提交还原协同过程。

提交清单对照

比赛要求	本项目落地情况
Public 仓库 + 细粒度提交历史	GitHub Public，253 个独立 commit（截至 2026-09-30），禁止一次性打包
AI 对话历史 Prompt 快照	ai-prompts/ 目录，30 份快照 + 索引，与代码分开提交
README 复现指南	仓库根 README.md（环境 / Key 两种配法 / 启动 / 自检 / 演示话术）
5-8 分钟演示视频（含 AI 协同）	docs/演示视频分镜.md（逐镜 + 解说词，本地录屏）
技术文档 PDF ≤30 页	本文件
开源协议	MIT License（仓库根 LICENSE）

1. 项目背景与定位

学生在校园里要同时盯课表、待办、取件、各类咨询，信息散落在不同系统。校园时间管家用「一个助手」把这些收口，分三层：

- 学生活端（旗舰）：课表时间规划 + 校园问答 + 快递外卖查询。学生说一句话（如「把游泳排到周四 19:00」），系统算出提案、弹确认卡，点确认才写进日程。
- 驿站 / 商户侧：监听快递外卖台账变动，主动推送取件提醒与滞留预警，用自定义人格智能回复咨询。
- 管理控制台：零代码维护知识库、客服人格，并在线配置大模型本身。

三层共享同一个 Agent 引擎（判定 + 协调执行）+ 知识库 + 大模型，避免重复实现。

2. 技术栈与运行环境

层	选型	说明
后端	Python + FastAPI	把函数变成网页可调用接口；uvicorn 启动
前端	原生 HTML / JS	无需打包工具，不引入前端框架；确认卡走内嵌 /confirm 页面
大模型	DeepSeek API	Key 走环境变量 / .env 或管理后台，不写进代码
Agent	自写轻量 ReAct 循环	不套重框架；确定性分支优先于模型
存储	JSON 文件（app/data）	由 CAMPUSTIME_DATA_DIR 指向，测试用临时副本
测试	FastAPI TestClient + 自写 Checker	无需 pytest，评委零额外安装即可跑

运行（复现摘要）

```
pip install -r requirements.txt
cp .env.example .env # 填入 DEEPSEEK_API_KEY（或启动后在 /admin 在线填，免重启）
uvicorn app.main:app --reload
# 浏览器打开 http://127.0.0.1:8000
python tests/run_all.py # 全量自检：通过 1398 项，失败 0 项（共 17 批）
```

3. 系统总体架构

一次「学生说话 → 出提案 → 确认写库」的请求流：

```
学生输入
 └──> /api/chat
 ├── 【判定层】先分「问 / 办」，再定模块
 │ ① 样本库模糊匹配 → ② 语义层问一次 LLM 只要 {module,intent}
 │ ③ 关键词表兜底 （前端也可显式指定 module）
 ├── 确定性分支(3/3a/3b/3c/3d/3c-ter) 命中即出提案，不调模型
 │ ├── is_confirmation+peek_pending 学生回「确认」→ 系统写/删库
 │ ├── wants_clear_timetable 出清空提案+确认卡
 │ ├── wants_add_todo 出待办提案+确认卡
 │ ├── wants_list_todo 纯读，列出待办
 │ └── missing_thing_of 『看不见 X』真查库核实
 └── 都没命中 → 模型(ReAct) 用 read-only 工具产出提案
提案 = options[]，每条带系统盖章的 kind
 └──> 前端 renderOptions 按 kind 渲染确认卡
 └──> 学生点确认 → 系统写库（模型全程不参与）
```

关键点：写库的唯一入口是系统侧的确认动作，模型无论走哪条路都碰不到写入接口。这是「学生没点头就宣布已经删了/加了」这一类事故的硬防线。

3.1 一次完整请求的时序（以「帮我把游泳排到周四」为例）

阶段	发生什么	谁在做	写库?
① 判定	先分「问 / 办」，再定模块；同一次判定给出 {module=planner, intent=add_todo}	判定层（样本库 → 语义层 → 兜底）	否
② 算账	读周表，避开已有课程与待办，摊出 2-3 个可行时段	执行层（planner 确定性代码）	否
③ 出卡	写成 options[]，系统盖上 kind，存入待确认暂存柜（TTL 3 小时）	执行层	否
④ 确认	前端按 kind 渲染确认卡；学生点【确认】	前端 + 系统	此时才写
⑤ 回执	从暂存柜取出提案执行，回「已安排」，周表/月表立刻可见	执行层	已写

注意第 ① 步和第 ③ 步是两件独立的事：判定层只负责「这句话想干什么」，它给出的结论会被喂给执行层那套老代码，而它自己一步都跨不进执行层。

4. 核心设计一：Propose / Execute 分离

为什么

早期用纯模型直接回「已经帮你排好啦」，学生刷新日程却空空如也——模型没有写库能力，只是在文字里「表演」写库。把「说要做」和「真做了」混在一起，学生无法区分，是最伤信任的故障。

怎么做

- Agent 只调用 read-only 工具（查空档、列待办、读文件）。任何会改变状态的操作，都先变成一张「提案」返回，绝不在对话轮里直接落库。
- 提案由系统盖章 kind（如 todo_add / todo_remove / todo_mode / todo_slots / timetable_clear）。前端只按 kind 渲染对应确认卡——显示与否由系统决定，不由模型自报。
- 提案先存进待确认暂存柜（app/agent/pending.py：内存 + 磁盘双写、3 小时 TTL）。学生点【确认】后系统才取出来执行；点【取消】或过期则一切不变。

「给 Agent 的提案做标记、前端识别后渲染」的落地形态是 kind，且必须由系统盖，不能由模型盖（否则「显不显示弹窗」的决定权又回到模型手里，正是「经常不弹弹窗」的成因）。

5. 核心设计二：判定层 / 执行层分离

本项目最重要的一条设计，也是它和「随便套个 LLM 问答」的根本区别。

5.1 它要解决的问题

学生在报障时说了一句一针见血的话：

> 「很多的字都是接不住的，你现在能接住的都是我测试给的。」

过去每报一次障就往词表里加一条：planner.py 里堆到 30 多张关键词/正则表，router._QUICK_MAP 还有 50 多个硬编码词，全是 if keyword in text 这种朴素子串匹配。这是在拿字符串匹配干语义理解的活——中文口语的写法是无限的，「你说一句我补一词」永远慢一拍，词表加多长都收敛不了。

5.2 解法：把「听懂」和「执行」切开

```
┌── 判定层（可换、可兜底、可以交给模型）──────────────────┐
| ① 样本库 历史成功原话 Dice 模糊匹配（零成本、离线可跑） |
| ② 语义层 问一次大模型，只要一个 JSON {module, intent} |
| ③ 规则兜底 30 多张词表 + _QUICK_MAP（只在 ①② 都没结果时才轮） |
```

```
└──────────────────────────────────────────────────────────────────────────────────────────────────────────┘
┌── 执行层（系统独占，模型永远碰不到）──────────────────────────────────────────────────────────────┐
│ 查空档 · 排期 · 出候选卡 · 写库 │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

5.3 为什么这条分离没有破坏 Propose / Execute 的地基

语义层那个大模型干的活只有一件：把这句话听懂，输出结构化意图。它拿不到任何写库接口，也碰不到排期算法。拿到 JSON
之后，查空档、摊候选、出卡、写库走的还是原来那套确定性代码。所以「AI
只提案、系统才执行」一条不破——变的只是「听懂人话」这一步不再由正则独占。

5.4 2026-09-27：顺序倒成「语义优先、关键词兜底」

档	做什么	成本	没配 Key 时
① 样本库	历史成功原话 Dice 模糊匹配（阈值 0.42，词长 <4 不匹配）	零	照样能用
② 语义层	问一次 LLM，只要 {module, intent}，置信度 <0.6 丢弃	一次调用	自动跳过
③ 规则兜底	词表命中就走	零	兜底

此前是「关键词第一、语义兜底」，结果是判定权仍握在词表手里，学生一换说法就掉回模型。倒过来之后，类型（module）与意图（intent）由同一次判定一起给出，`_QUICK_MAP`
从「第一判据」降为「兜底」。

5.5 为什么语义层可以反过来复核规则

规则判了「要办事」，可这句话其实是个问句，或者人就在问答型栏目里（校园问答 / 快递 /
外卖，不含 planner）——这时让语义层复核一次。但否决闸门只在两种正面肯定下才生效（这句带提问信号，或语义层明确说 chat），不是「结论不同就否决」：样本库是模糊匹配，误否决一张本来就该出的卡，比漏判更让学生恼火。多义词（「安排」既是名词又是动词）是关键词表的天生盲区，补词补不好，用这招才治得住。

取向：宁可漏判掉回老路，不可替学生办错事。白名单之外不收、置信度不够不要、没配 Key
就安静返回 None。

6. 确定性意图路由（/api/chat）

为减少「模型自由发挥」导致的不可控，能在代码里定死的意图都收归确定性分支，模型只处理真正开放的问答。

分支	触发	系统行为	写库?
确认执行	is_confirmation + 有 pending	按暂存提案写/删库	是（确认后）
清空课表	wants_clear_timetable	出清空提案 + 确认卡	否（等确认）
加/删/改待办	wants_add_todo / remove / retime	算时间 + 出确认卡	否（等确认）
读待办	wants_list_todo	list_todos() 真查并念出	否（纯读）
看不见 X	missing_thing_of	真查库核实，在/不在如实说	否

「不弹弹窗」的根因通常是：一句话没被任何确定性分支接住、掉进模型，模型只回纯文本（无 options/kind），确认卡自然不来。因此扩大确定性分支的覆盖面是修复此类问题的主战场；而扩大覆盖面靠的是判定层，不是继续加正则。

7. 关键迭代：三次架构级改动（2026-09-26 / 09-27 / 09-30）

7.1 第一次：引入判定层（09-26）

问题：学生反馈「很多的字都是接不住的，你现在能接住的都是我测试给的」。研判确认 planner.py 已堆 30 多张词表、router 里 50
多个硬编码词，属于「用字符串匹配干语义理解的活」，加词无法收敛。

文件	改动
app/agent/intent.py	新增：意图白名单 + 问句识别 + 样本库（Dice 模糊匹配）+ 语义层（LLM 只读意图）
app/agent/router.py	关键词落空时先查样本库，再问语义层
app/main.py	/api/chat 接入语义兜底，规则接不住时不再直接掉模型
tests/test_08_intent_fallback.py	新增第八批护栏，覆盖样本匹配、语义闸门、降级路径

7.2 第二次：类型判定整体重做（09-27）

起因：学生截图反馈「校园问答」栏问「我要去哪里取快递」竟出了待办候选卡。第一次修复只堵了那一句话，学生随即指出「你只是修了这一个，其他类型的区分没改」——问题不在那一句，而在「归哪个模块」的判定方式本身。

研判：「归哪个模块」由 router._QUICK_MAP 的 50 多个子串匹配说了算，「想干什么」另由 30 多张表负责，两处互不通气。于是出现「模块是校园问答、意图却是加待办」这种自相矛盾的判定。

改动：把类型也收进判定层，与意图做同一次判定，顺序倒成「语义优先、关键词兜底」；/api/chat 的判定层前移到最前，五条确定性分支共用同一次判定；否决闸门全分支让路；新增 QUESTION_RE / looks_like_question / _QA_MODULES——先分「问 / 办」，粗信号只决定要不要复核、不下结论。

两次自踩自修（值得记录）：

- 语义层把口语翻成规范话喂老解析器时，规范话把原话里的信息丢了（「帮我安排周二的游泳」里的「周二」不在语义字段中 → 候选摊满一整周）。修正：规则认得的句子要用原话解析，规范话只在规则读不出来时兜。
- 模块顶部 from X import Y 是导入时绑定的：Router 在 __init__ 里建死客户端后，打桩/换配置都传不进来，语义层会安静地退回关键词表（异常被 except 吞掉）。修正：按需在函数内 import 才看得到补丁。

7.3 第三次：错别字纠正与思考过程透明化（09-30）

起因：学生报障截图一句话——「没有展示思考过程，没有修改错别字」。他打的是「中文12点的车」（「中文」是「中午」的笔误），管家回的卡片写着「中文的车」，错别字一字未改，而且「12点」也整段丢了。

研判：这句话只有两条，但每条都是一整类问题。查下去发现三个独立根因，其中第一个与「错别字」无关，却比错别字严重得多：

层	问题	后果
样本库	模糊匹配命中就把整条字段抄走	「我想去健身」撞上「…游泳…」(0.73) → 卡片写「游泳」；「练会儿钢琴
规范话	to_canonical 只拼 start and end	学生只说一个起点（「12点的车」）→ 整个钟点被悄悄丢掉 → 规范话退回
规则层	_normalize_clock 只认「读音对、写法对」的中文	「中文」不是时间词、擦标题擦不掉 → 整串留在标题里

改动：

- 样本库命中分两档（_SIM_FULL = 0.9）：几乎一字不差才算「全命中」、可复用字段；0.42~0.9 只是「像」，只借 module/intent、字段一律清空，要事名的意图继续问模型现读。模糊匹配只配定路线，不配定内容。
- to_canonical 补 elif start：单独的起点也拼进规范话（end 缺失交给 planner 按默认时长补）。
- 规则层新增 _fix_typo_time_words（时间词的形近笔误纠正），接在 _normalize_clock 开头，一处生效两条路（抠标题 + 抠时段）。三道闸门缺一不可：
- ① 位置上是时间词的先原样收下 —— 不然会连环改：「今田晚上有课」→「今夜里上午课」（今天 / 今晚 / 今夜互相都只差一个字）。
- ② 长得像时间词的正经词不许动（_NOT_TIME_WORDS + 星期几）—— 「上班」和「上午」只差一字，不拦就会把「加个上班打卡」纠成「上午」再擦掉、变成「打卡」，悄悄少一个词，比报错难发现得多。
- ③ 必须处在时间语境（后面紧跟钟点或星期）才纠 —— 取向是宁可漏纠：没钟点的笔误（「明天交电费」）交给语义层，这条兜底只为「离线演示也别把错字写进日程」。

思考过程怎么露出来：/api/chat 改成薄壳，主体移入 _chat_impl(req, meta)，在出口统一插一条「我把这句听成 办事 / 问事｜依据：哪一档｜事名读成了什么」。为什么必须单出口 —— _chat_impl 有一千多行、几十个 return，挨个 return 去加必然漏（实测就漏了「追问」那条，而那正是学生说错话时最该看到解释的）。meta 走参数而不用全局量：同一时刻可能有多个请求在跑，全局量会把 A 学生的判定写进 B 学生的回复。

学生端把轨迹默认摊开（可在设置里关）——这是学生核对「它有没有听错」的唯一凭据，并在渲染时过滤三处噪声：空的 ReAct 流程标记、与正文重复的「最终回答」、超长的 answer 截断。顺带把 /classify 的 source_note 与思考轨迹统一到同一张「人话表」（原来各写一张，必然有一处会漂）。

7.4 验证

- 新增/扩充回归：第八批覆盖类型判定重做（问句识别、语义压过关键词、有/无 Key 双路、端到端不出卡）。
- 把学生截图里那两句原话（「我要去哪里取凯迪」「我要去哪里取快递」）钉成永久回归，防止复发。
- 全量 tests/run_all.py：通过 1398 项，失败 0 项（共 17 批）。

8. 测试策略

测试不依赖 pytest，只用项目虚拟环境已有的 FastAPI TestClient + 自写 Checker，评委零额外安装即可跑。

批次	覆盖内容
test_01 基础	健康检查、页面路由、知识库 CRUD、客服人格、通知、没配密钥时不裸 500
test_02 上传	上传与边界防护、掩码回显与「掩码串不被写回」防呆、docx/xlsx/pptx 正文与表格解析
test_03 规划	周表导入与校验、空档避课、待办防撞课、月表聚合、会话历史上限
test_04 人设	三套性格接口、规划 Mock「提议→确认→写入」闭环、性格腔调真注入
test_05 学生视图	只读接口数据契约、主面板四视图标记、既有接口回归护栏
test_06 加东西	提案工具直调、日程面板提案、沙箱隔离
test_07 待办意图	三类「代办」口语走系统分支、空列表分支

test_08 意图兜底	样本库匹配、语义闸门、**类型判定整体重做**、残卡字样永久回归
test_09 增删查改	点名删/盲删、改期换时间、语义层直出卡、确认后落库与旧条移除
test_10 标记完成	标记完成改由系统接管（撤掉模型直调写工具）、三分支 ok/many/empty、问答栏不再静默失败
test_11 待办定位	「第N条 / 裸数字 / 连写23 → [2,3] / 最后一条」、有清单时永不返回 empty、撞时间批量删
test_12 批量删	「都删了」指代上一句清单、批量确认条（一张条一次点头）、「每个删一个」去重（与全删相反）
test_13 猜测与证据	上下文四级台阶、只认结构性事实（精确同名 / 候选业务等价）、不做相似度猜测
test_14 语义强弱	语义层单独猜的 done 是弱信号（让给删待办）、规则层硬信号照旧；打桩必须打到 app.main 模块名
test_15 口语样例	20 句真人口语自主探测（软删/错字/语气词/指代/抱怨/知识库错字）+ 沙箱假绿的暴露
test_16 错别字与轨迹	时间词形近笔误的三道闸门、样本库两档命中、规范话不丢起点、思考轨迹必在（含追问/问答两分支）
test_intent_classify	判定层诊断口 /classify 62 项：问/办、模块与意图同判、有/无 Key 双路、只读红线

- 每个测试文件在独立子进程 + 临时数据副本（sandbox）中运行，绝不污染真实演示数据。
- CI 式总账：run_all.py 汇总每批「通过 X 项，失败 Y 项」，有失败退出码为 1。
- 血泪教训已固化为自检手法：沙箱只套一半等于没套。曾有一个测试上半段露在沙箱外，每跑一次全量就往真实暂存文件写一张确认卡，界面上凭空多出待办。现在跑测试前后给几个运行时文件算 md5 对比，必须一字不变。

```
python tests/run_all.py
# 总账：通过 1398 项，失败 0 项（共 17 批）全部通过
```

9. AI 协同开发与交付流程

本项目全程在人与 AI 协同下完成。为满足「细粒度提交 + Prompt 快照」要求，约定：

- 每做一步逻辑改动就单独 commit，绝不在最后一次性打包（本文档版本日为 253 个，此后持续增加）。
- 每次开发会话写一份 Prompt 快照（ai-prompts/YYYY-MM-DD-<主题>.md），记录「用户 Prompt → AI 研判 → 改动 → 验证」四段。
- 快照文件与代码分开提交（单独 commit），评审把任意快照与其 commit 对照即可还原那一步协同过程。
- 当前共 30 份快照，索引见 ai-prompts/README.md。

9.1 交付链路本身也被记录

仓库推送到 GitHub 的过程同样留了快照（ai-prompts/2026-09-27-github-push-and-contributor-identity.md）：三重独立卡点（代理拦截 → 仓库未创建 → 凭据缺失）的定位与解法，以及「提交归属到底由什么决定」的取证过程——commit 里只有 name 和 email 两行自由文本，GitHub 只按 email 反查账号、完全忽略 name。这类基础设施环节的踩坑与推演，同样属于「AI 协同过程」的一部分。

9.2 协同模式的两个特征

- 先取证再动手：每次改动前先读代码、跑测试、考证具体是哪一行在起作用，避免「症状相同就套上次的修法」（同一句报障在本项目报过 4 次，每次根因都不同）。
- 架构级判断与补丁级修复分开：当学生指出「你只是修了这一个」时，不是再补一个正则，而是回到判定方式本身重做（见第 7 节）。

10. 复现指南（README 摘要）

- 环境：Python 3.10+（本项目在 3.13 上开发与测试），pip install -r requirements.txt。
- 密钥（两种任选）：① 启动后在 /admin 的「API 配置」在线填，立刻生效免重启；② cp .env.example .env 填入 DEEPSEEK_API_KEY。settings.json 已在 .gitignore 排除。
- 没配 Key 也能跑：自动切到确定性 Mock 模式，排程交互、卡片渲染、知识库问答等离线链路照常演示。
- 启动：uvicorn app.main:app --reload，打开 http://127.0.0.1:8000。
- 自检：python tests/run_all.py（临时数据副本，不碰演示数据），预期 1398 项全绿。

演示话术（都能直接试）

你说	应该看到
帮我把复习线性代数安排到周四 19:00-20:30	出确认卡；点了确认日程里才出现（没确认不写）
我的代办呢	直接念出真实待办列表（真查库，不是编的）
把周四那个瑜伽挪到周五	出改期卡，确认后旧条目移除、新条目落位
健身显示不出来	系统真去查库核实，如实回答「在的 / 不在」
我要去哪里取快递	走校园问答检索知识库回答，不会被误当成「加个待办」

11. 局限与后续

- 图片/扫描件读不出字：需要接 OCR；目前只支持 txt / pdf / docx / xlsx / pptx 的文字层。
- 没有真正的账号体系：演示级身份切换，无密码；app/data/student/ 是「一个人的」个人库，未按学号隔离。
- 长资料会被截断：单次最多给模型 6000 字，未做分块检索。
- 周期性/跨天待办未做：现在一条待办只属于某一个日期。
- 确定性覆盖面仍可扩大：判定层的语义闸门偏保守（宁漏不误），个别口语说法仍会掉回模型反问。
- 演示视频需本地录屏：逐镜脚本见 docs/演示视频分镜.md。

模型仅在开放问答时启用；高可靠操作（写 / 删 / 改）一律系统执行，这条边界不会为了效果而放松。

附录 A：接口清单（39 个 API）

7 个页面路由：/（登录）、/student、/plan、/settings、/confirm、/admin、/station。全部接口如下（只读接口绝不写库；写库接口全部要求携带系统侧的确认动作）。

接口	作用
GET /api/health	健康检查
POST /api/chat	主对话入口：判定层 + 确定性分支 + 模型兜底
POST /api/chat/reset	重置某个会话的历史
GET /api/conversation/{sid}	读会话历史（前端刷新后恢复聊天记录）
GET /api/pending/{sid}	读待确认提案——前端据此渲染确认卡，只读不执行
GET /api/timetable	读学生周表
POST /api/timetable/apply	应用课表（整表替换，需先经过预览确认）
POST /api/timetable/clear	清空课表

GET /api/todos	读待办列表
GET /api/todos/month	月表聚合视图
POST /api/todos	新增待办（确认后）
POST /api/todos/batch	批量写入待办
PUT /api/todos/{tid}	修改待办
DELETE /api/todos/{tid}	删除待办（确认后）
POST /api/todos/mode	保存日程展示模式
GET /api/express	快递只读视图（含实时状态标签），供卡片面板用
GET /api/takeout	外卖只读视图，供卡片面板用
GET /api/student-personas	三套学生助手性格预设
POST /api/student-message	学生每轮对话上报（管理员收件箱数据源）
GET /api/notices	学生端读通知
POST /api/upload	上传文件并提取文字
GET /api/uploads	上传文件列表
GET /api/uploads/{fid}	读某个文件的提取文字（AI read_uploaded_file 走这里）
GET /api/uploads/{fid}/download	下载上传原件
GET /api/workorders	读学生工单
POST /api/workorders	提交工单（学生确认上报后才生成）
POST /api/workorders/{wo_id}/status	修改工单状态（管理端）
GET /api/persona	读驿站客服人格
POST /api/persona	写驿站客服人格
GET /api/admin/kb	列出知识库
POST /api/admin/kb	新增知识库条目（零代码维护）
PUT /api/admin/kb/{index}	修改知识库条目
DELETE /api/admin/kb/{index}	删除知识库条目
GET /api/admin/settings	读大模型配置（密钥掩码输出）
POST /api/admin/settings	写大模型配置（掩码串防呆，不覆盖真密钥）
POST /api/admin/settings/test	在线连通自检
POST /api/admin/notify	发布通知（支持附件）
GET /api/admin/messages	学生消息收件箱
POST /classify	判定层诊断口（只读）：摊开 is_question / module / intent / confidence / 是哪一档判

附录 B：数据文件

B.1 种子数据（有意入库，管理台可零代码增改删）

文件	内容
app/data/courses.json	示例课表，旗舰模块的演示数据
app/data/kb.json	校园知识库（图书馆/食堂/校车/校医院/宿舍/奖学金…）
app/data/express.json	演示快递台账（含一件「滞留 5 天」件，用于预警演示）
app/data/takeout.json	演示外卖订单（制作中/配送中/待取）

B.2 运行时数据（不入库，已在 .gitignore 设防）

文件	内容	为什么排除
settings.json	管理后台填的模型密钥/地址/模型	含真实密钥
persona.json	驿站客服人格	运行时数据
notices.json	管理员发布的通知	运行时数据
student_messages.json	学生对话上报记录	含隐私
uploads.json + uploads/	上传文件元信息与实体	含隐私
workorders.json	学生工单	含隐私
pending.json	待确认提案暂存（TTL 3 小时）	运行时状态
intent_samples.json	判定层样本库（累积的成功判定样本）	运行时数据
student/timetable.json	学生周表	个人数据
student/todos.json	学生待办	个人数据
student/sessions.json	会话历史（最多 20 条）	个人数据

数据目录可用环境变量 CAMPUSTIME_DATA_DIR
指向别处——这正是自动化测试的数据沙箱开关：测试把 app/data
复制到临时目录再跑，跑完删掉，真实演示数据一个字节都不动。

— 文档结束 —